Kavach AI

Anti-Digital-Arrest & Scam Conversation Shield

Prototype Requirements Document — Hackathon Round 2 Submission

Team name: _____ Members: _____

Working Links (all three)

1. Live Working Prototype Streamlit web app: text, audio, voice recording, link checker	https://YOUR-LIVE-APP-LINK-HERE
2. Demo Video Explains the problem, the solution and a live walkthrough	https://YOUR-DEMO-VIDEO-LINK-HERE
3. Source Code (GitHub) Full code, README, 42 automated tests	https://YOUR-GITHUB-REPO-LINK-HERE

Each link opens the prototype, the explanatory demo, and the code. Tap or click to open.

1. The Problem

Phone and video-call scams in India, especially **"digital arrest"** fraud, rely on **psychology, not hacking**. A caller poses as police, CBI, customs or a courier company, says the victim is under investigation (a drug parcel, money laundering, a linked Aadhaar), and keeps them on the line while demanding money or an OTP. Real agencies never arrest anyone over a call.

- **Victims panic and cannot think** in the moment: threat + secrecy + a deadline is designed to switch off judgement.
- **Existing tools miss it.** Spam-call blockers look at the number; antivirus looks at apps. Neither understands *what is being said*.
- **Language gap.** Scammers speak English, Hindi and Hinglish (Roman-script Hindi). Most tools work in English only.
- **The most vulnerable** (elderly, first-time digital users) need something simple, in their language, that gives a clear action: stop.

2. Our Solution and How It Solves the Problem

Kavach AI listens to the *content* of a conversation (typed, uploaded audio or a voice recording), transcribes it locally when it is speech, and scores it against the **five classic scam-pressure tactics** plus credential theft. It then tells the user in plain language **why** it is suspicious and **what to do next**.

Problem	How Kavach AI addresses it
Victim cannot judge in the moment	Instant 0-100 risk gauge and a red **STOP** banner at score 50+, with the exact phrases highlighted in the text.
Tools ignore what is said	Analyses the conversation itself: authority claims, threats, secrecy, urgency, money demands, OTP/PIN requests.
Hindi / Hinglish gap	Rules handle English, Roman Hinglish and Devanagari Hindi; local Whisper speech-to-text supports Hindi and English.
No clear next step	One-tap **Call 1930**, **Alert a family member** (WhatsApp pre-filled message), cybercrime.gov.in, and a downloadable report.
Scam links by SMS/WhatsApp	Offline link checker flags fake bank/government domains, shorteners, IP links and .apk downloads without opening the link.
Privacy fears about call audio	Local transcription, nothing logged, session text wiped when the call ends (see Section 6).

3. Key Features and Functionality Demonstrated

Feature	What it does in the prototype	Source
Text analysis	Paste English / Hindi / Hinglish conversation. Ready-made samples (digital arrest, bank OTP, son in custody, normal call) for instant demo.	app.py, detector.py
Audio upload	Upload wav/mp3/m4a/ogg/flac/webm, transcribe locally, edit the transcript, then analyse.	app.py, transcriber.py
Voice recording	Record a short clip in the browser, transcribe and analyse it.	app.py
Explainable score	Risk gauge, level (Lower signal / Caution / High / Very high suspicion), signals with points and the matched phrases highlighted.	detector.py
Smart negation	"Never share your OTP" is treated as a *warning*, not a demand, so genuine bank advice is not flagged.	detector.py
Action buttons	Call 1930, WhatsApp alert to family, cybercrime.gov.in, download .txt report.	app.py
Link checker	Scores URLs 0-100 with reasons; lookalike gov.in domains are not trusted; link is never opened.	linkcheck.py
Learn tab	Six common Indian scams (digital arrest, KYC, parcel, investment, QR/refund, relative in trouble) and the golden rule.	app.py
Accessibility	Large-text toggle and Hindi UI toggle for elderly users.	app.py
REST API (for a mobile app)	/analyze, /check-link, /transcribe, /health endpoints with optional API-key auth.	api.py
Live call-session API	Anonymous device token, running score during a call, one-time alert, history, privacy-safe community number reports.	calls.py, db.py

4. Demo Walkthrough (what the video / live demo shows)

Suggested 3-minute flow so the demo explains the **problem** and the **solution** end to end:

Step	Action	What the judges see
1 (0:00)	State the problem in one line: "Digital-arrest scammers keep victims on the line and pressure them into paying."	Problem framing, Section 1
2 (0:20)	Text tab: click the **Digital arrest (Hinglish)** sample, press Analyze.	Gauge near 100, STOP banner, five signals with highlighted phrases
3 (0:50)	Click **Normal call** sample.	Score 0, green box: no false alarm
4 (1:10)	Record tab: say a scam line in English or Hindi; transcribe and analyse.	Local speech-to-text feeding the same detector
5 (1:50)	Show the action buttons: Call 1930, Alert a family member, download report.	Clear next step, not just a score
6 (2:10)	Link tab: paste `https://sbi-kyc-update.xyz/login`.	High risk with reasons; official sbi.co.in scores 0
7 (2:35)	Mention the API and call-session privacy design; close with limitations.	Path from prototype to a mobile product

5. Technology and Approach

Approach: an **explainable, rule-based** detector, not a black box. Every point in the score maps to a named signal and visible evidence, which matters when advising a frightened person. Speech recognition is separated from detection, so any transcript source can feed the same engine.

Layer	Technology	Role
Frontend	Streamlit (Python), custom CSS, mobile-responsive	Five-tab UI: Text, Upload, Record, Link, Learn
Detection engine	Python regex rule sets, Unicode NFC normalisation	Six signal groups across English, Hinglish, Devanagari; negation handling for warnings
Speech-to-text	faster-whisper (Whisper, default "small" model), int8 on CPU	Local transcription of Hindi / English audio, no cloud call needed
Link analysis	Python, urllib; heuristics only	Offline URL risk score; never fetches the URL
Backend API	FastAPI + Uvicorn, Pydantic validation, CORS	REST endpoints for a mobile client; upload size limits (25 MB / 5 MB chunks)
Storage	SQLite (standard library), SHA-256 hashing	Device tokens, call sessions, hashed number reports
Quality	pytest, FastAPI TestClient	42 automated tests covering detector, links, API, and call sessions

Risk scoring (fixed points per signal group, capped at 100)

Signal group	Points	Example it catches
Authority impersonation	15	"I am Inspector ... from Crime Branch", "digital arrest"
Arrest / legal / account-block threat	25	"you will be arrested", "your son is in custody", "account will be blocked"
Secrecy / stay on the call	20	"do not tell your family", "do not disconnect", "kisi ko mat batana"
Urgency / pressure	15	"immediately", "turant", "within 30 minutes"
Money-transfer demand	25	"transfer money to a safe account", "paise transfer karo"
OTP / PIN / password / remote-access request	45	"OTP bhejo", "install AnyDesk" (Roman, Devanagari and mixed spellings)

Levels: 0-24 Lower detected signal, 25-49 Caution, 50-74 High suspicion, 75-100 Very high suspicion. The score is a heuristic count of pressure tactics, **not a probability**, and the app says so on screen.

System flow

Input	Speech-to-text	Detection engine	Output
Typed text Uploaded audio Mic recording Pasted link	faster-whisper (local, Hindi / English)	Rules + negation Score 0-100 Evidence phrases	Gauge + STOP banner Why it is risky Call 1930 / alert family Report download

Audio goes through speech-to-text first; typed text and links go straight to their analysers. The same engine also powers the REST API.

6. Privacy and Responsible Design

- **Local processing:** speech is transcribed on the machine running the app; no third-party speech service is used.
- **Transcripts are wiped** when a call session ends, and abandoned sessions are purged after one hour, unless the user explicitly opts in to store text.
- **Anonymous by design:** devices get a random token (stored only as a hash); no name, phone number or contact list is needed.
- **Phone numbers are never stored in the clear:** community scam-number reports use a salted SHA-256 hash, and a number is flagged only after **3 reports from distinct devices**, which blocks one person from falsely labelling a number.
- **Link checker never opens links**, so users cannot be exposed to a malicious page by checking it.
- **Honest framing:** every result carries a disclaimer that a low score does not prove safety.

7. Validation

The prototype ships with **42 automated tests, all passing**: 27 detector tests (real screenshot-based scam transcripts, garbled speech-to-text output, Hinglish and Devanagari variants, and *normal-speech false-positive checks*), 6 link-checker tests, 4 API tests and 5 call-session privacy/isolation tests. Run `pytest -q` to reproduce.

8. Current Limitations and Next Steps

Prototype limitation (stated openly)	Planned next step
Rule-based: can miss new scripts or flag unusual normal speech; score is not a probability.	Collect consented, anonymised examples to tune rules, then add an ML classifier alongside the rules.
Browser microphone cannot tap real phone-call audio; speakerphone near the device is needed.	Native Android app using the existing call-session API for in-call, real-time alerts.
Whisper may write Hinglish in Devanagari; first run downloads the model.	Hosted or on-device speech model for faster, lighter transcription.
Link check is offline and heuristic only.	Add reputation feeds and the community-reported number database at scale.

Bottom line: Kavach AI is a working, explainable first line of defence that turns "something feels wrong" into a clear, in-language instruction: **stop, do not pay, do not share an OTP, call 1930**. A production-ready system is not claimed; this prototype demonstrates the core idea end to end.

Submission Links (repeated for convenience)

1. Live Working Prototype Streamlit web app: text, audio, voice recording, link checker	https://YOUR-LIVE-APP-LINK-HERE
2. Demo Video Explains the problem, the solution and a live walkthrough	https://YOUR-DEMO-VIDEO-LINK-HERE
3. Source Code (GitHub) Full code, README, 42 automated tests	https://YOUR-GITHUB-REPO-LINK-HERE